MODULE 17 · SECTION 2 - INGESTING AND PROCESSING THE DATA

Low-code, SQL Pipelines and AI Enrichment

Cloud Data Fusion, Dataform, BigQuery data preparation, and enriching data with AI in SQL or in Dataflow.

SECTION 2 of 5 EXAM WEIGHT 25% MODULE 17 of 45

admin@sthithapragna.com

Choose the tool by **who will maintain it**, then by where the data already lives.

Six tools for this module

TOOL	WHO IT SUITS	WHAT IT DOES
Cloud Data Fusion	Integration teams, low-code	Visual pipelines across many sources
Dataform	Analysts and engineers who write SQL	SQL pipelines in BigQuery, in Git
BigQuery data preparation	Analysts, no code	Gemini-suggested cleaning steps
Dataprep by Trifacta	Partner product	Visual cleaning, in older questions
BigQuery AI functions	SQL users	Call Gemini models from SQL
Dataflow Enrichment, RunInference	Pipeline engineers	Lookups and model calls in a stream

THE TIEBREAKER

The first two columns decide most questions. The tools overlap; the people do not.

admin@sthithapragna.com

Visual pipelines, run on Spark

- Built on open-source **CDAP**. Pipelines are drawn as a graph of **plugins**.
- Each run gets a **short-lived Spark cluster**, created and deleted for you.
- **Private instances** through Private Service Connect or VPC peering.
- **CMEK** in every edition, set at creation, covering resources plugins create.

> REMEMBER
>
> The CMEK key must be regional, in the instance's region, and applies only to new resources.

Wrangler: see the data, then clean it

- Interactive cleaning on a **sample**, the first **1,000 records**.
- Each step is a **directive**. The ordered list is a **recipe**.
- **Batch pipelines only**. Wrangler does not stream.
- Turn the recipe into a **pipeline**, or the work is not kept.

> THE SIGNAL
>
> Wrangler is Data Fusion's answer to 'analysts must clean data visually before it loads'.

Dataform: software engineering for SQL

- **SQLX** files. **ref()** links them, so Dataform works out the order.
- Code lives in **Git**: Dataform's own, or GitHub, GitLab, Azure DevOps, Bitbucket.
- **Development workspaces** are private copies. **Compilation** turns SQLX into plain SQL.
- Each repository runs as its own **custom service account**.

> THE SIGNAL
>
> 'Version control, code review, dependency order, tests' in a SQL question means Dataform.

An incremental table for Larkspur's readings

```
1  config {
2    type: "incremental",
3    uniqueKey: ["meter_id", "reading_ts"],
4    protected: true,
5  }
6  SELECT meter_id, reading_ts, kwh, received_ts
7  FROM ${ref("meter_readings")}
8  WHERE kwh IS NOT NULL
9  ${when(incremental(),
10   `AND received_ts > (SELECT MAX(received_ts) FROM ${self()})`)}
11 QUALIFY ROW_NUMBER() OVER (
12   PARTITION BY meter_id, reading_ts ORDER BY received_ts DESC) = 1
```

3-4 Resends merge on the key. A full refresh is blocked.

9-10 Later runs read only rows received since the last run.

THE DETAIL

Filtering on received time still catches meters that upload days late.

Release or workflow configuration?

RELEASE CONFIGURATION

How to compile

- Which branch or commit
- Project, prefixes, suffixes, variables
- One per environment: dev, test, prod
- Produces a compilation result

WORKFLOW CONFIGURATION

When to run

- Runs a release's compilation result
- On a schedule, by tags or actions
- Overlapping runs are skipped
- Or run from Airflow, Workflows, Cloud Build

THE TIEBREAKER

Release says what and where. Workflow says when.

Incremental tables, precisely

- **First run**: the whole query runs, and builds the table.
- **Later runs**: the incremental filter is added, so only new rows are read.
- **uniqueKey** merges matching rows instead of appending them.
- Change the query, and you need a **full refresh**. **protected: true** blocks one.

> THE TRAP
>
> Protect tables whose source no longer holds the full history, or a refresh deletes the past.

BigQuery data preparation, and Dataprep

- **BigQuery data preparation**: Gemini suggests steps, and shows the SQL for each.
- Steps include filter, **deduplicate**, join and **validation**, with failures sent to an error table.
- Write modes: full refresh, append, **incremental**, **upsert**. **Dataform** schedules them.
- **Dataprep by Trifacta** is an Alteryx partner product. Recognise it in older questions.

> THE SIGNAL
>
> 'Analysts, no code, data already in BigQuery' points to BigQuery data preparation.

BigQuery AI functions

FUNCTION	WHAT IT DOES
AI.GENERATE	Any prompt, any data, optionally structured output
AI.GENERATE_TABLE	Extract fields into a schema you define
AI.CLASSIFY	Sort text into categories you list
AI.IF and AI.SCORE	Filter or rank rows with a plain-language condition
AI.EMBED	Turn text or images into embeddings
AI.COUNT_TOKENS	Estimate the input size before you pay for it

THE SIGNAL

Data already in BigQuery and a SQL team: enrich it in place, with no pipeline.

Enrich in SQL, or in the pipeline?

BIGQUERY AI FUNCTIONS

In SQL, over tables

- Data already in BigQuery
- Batch over many rows
- Analysts can own it
- Pay per model call

DATAFLOW

In the stream

- Enrichment: lookups in Bigtable, BigQuery, Feature Store
- RunInference: your own model in the workers
- Throttling and retries built in
- GPUs for heavy models

THE TIEBREAKER

Streaming, per-record, sub-second: Dataflow. A monthly batch of rows in BigQuery: SQL.

Enrich carefully

- Model output is **probabilistic**. Validate a sample against known answers.
- **Count tokens** before a big run. A prompt per row adds up fast.
- Store the **model and prompt version** with every result, so you can rerun.
- Send only the columns the model needs. **Personal data** rules still apply.

> THE TAKEAWAY
>
> Treat AI enrichment like any other transformation: tested, versioned and governed.

Phrases that pick the answer

THE QUESTION SAYS	REACH FOR
Low-code, many sources, integration team	Cloud Data Fusion
Clean **visually** before loading	Wrangler in Data Fusion
SQL with **Git, tests, dependency order**	Dataform
Only **new rows** each run, merge on a key	Dataform incremental table, uniqueKey
Classify or extract from text in BigQuery	BigQuery AI functions
Per-record lookup in a stream	Dataflow Enrichment transform

THE PATTERN

People first, then where the data lives, then how fresh the result must be.

> Larkspur's integration team must bring data from 15 business systems into BigQuery. The team builds pipelines visually and does not write code.
>
> Security requires every table and bucket the pipelines create to use Larkspur's own key in europe-west2.

Which approach fits best here?

A. A Dataflow pipeline in Java for each system, with CMEK flags on each job

B. Dataform SQLX files in Git, reading each system through external tables

C. A Cloud Data Fusion instance in europe-west2, with CMEK set at creation

D. A Cloud Run function per system, writing rows with the Storage Write API

◆ PAUSE AND DECIDE

Which approach fits best here?

A **Dataflow in Java**

Fifteen coded pipelines for a team that does not write code.

B **Dataform**

SQL over BigQuery. It cannot connect to fifteen outside systems.

C **Data Fusion with CMEK**

Visual pipelines, many connectors, and the key covers what plugins create.

D **Cloud Run functions**

Fifteen services to code and run, with no visual designer.

THE SIGNAL

Visual, many sources, no code, and CMEK on created resources: Cloud Data Fusion.

> Larkspur's curated readings table is rebuilt in full every night, scanning eight years of raw data. The raw table now expires partitions after 90 days.
>
> The team wants each run to process only new rows, merge resent readings, and never lose older history.

How should the Dataform table be defined?

A. As a view, so that it always reflects the current raw table

B. As a table that is rebuilt from scratch with each nightly run

C. As a scheduled query that appends every raw row each night

D. As an incremental table, with a uniqueKey and protected: true

◆ PAUSE AND DECIDE

How should the Dataform table be defined?

A **A view**

Every query scans raw, and history older than 90 days vanishes.

B **A nightly rebuild**

Rebuilt from 90 days of raw, it silently deletes older history.

C **Append everything**

Re-appends rows each night, so duplicates pile up.

D **Incremental, protected**

New rows only, resends merged, and no full refresh can erase the past.

THE SIGNAL

Only new rows, merge on a key, and a source that forgets: incremental, with uniqueKey and protected.

> Larkspur has about 2 million field-engineer job notes in a BigQuery table. Each month, analysts want every new note sorted into one of six fault categories.
>
> The analysts know SQL. There are no labelled examples and no machine learning team.

How should they classify the notes?

A Use AI.CLASSIFY in BigQuery, listing the six categories in the query

B Train a custom TensorFlow model, and serve it with Dataflow RunInference

C Write regular-expression directives for each category in Wrangler

D Use the Dataflow Enrichment transform with a Bigtable lookup table

◆ PAUSE AND DECIDE

How should they classify the notes?

A AI.CLASSIFY in SQL

The data is in BigQuery, the team writes SQL, and no training is needed.

B A custom model

Needs labelled data and an ML team, and neither exists.

C Wrangler regexes

Brittle patterns for free text, rewritten whenever wording changes.

D Enrichment lookup

A lookup matches keys. It cannot read and judge free text.

THE SIGNAL

Text in BigQuery, a SQL team, no labels: classify in place with AI functions.

> Each streaming reading must be enriched with the customer's current tariff before it reaches the live pricing view. The tariff table has 3.1 million rows.
>
> Tariffs change constantly as customers switch. The stream peaks at 10,000 readings a second.

How should the pipeline enrich each reading?

A Load all 3.1 million tariff rows as a side input, refreshed every hour

B Use the Dataflow Enrichment transform, with a Bigtable lookup per reading

C Call AI.GENERATE from BigQuery to work out the tariff for each reading

D Have a Cloud Run function query Cloud SQL for each individual reading

◆ PAUSE AND DECIDE

How should the pipeline enrich each reading?

A **A big side input**

Millions of rows on every worker, and up to an hour out of date.

B **Enrichment from Bigtable**

Fast key lookups, current data, with throttling and retries built in.

C **AI.GENERATE**

A model guessing a fact that sits in a table. Slow, costly and wrong.

D **Cloud SQL per reading**

Ten thousand queries a second on a database built for transactions.

THE SIGNAL

Large, constantly changing lookup, per record, in a stream: the Enrichment transform with Bigtable.

Module 17 in five lines

- **Data Fusion** for visual, many-source pipelines. **Wrangler** cleans a sample, batch only.
- **Dataform**: SQLX in Git. Release says what and where, workflow says when.
- **Incremental tables** read new rows; **uniqueKey** merges; **protected** blocks full refresh.
- **BigQuery AI functions** enrich in SQL. **Dataflow Enrichment** looks up per record.
- Validate, version and govern AI output like any other transformation.

COMING UP

Next: module 18, deploying pipelines with CI/CD, templates and environments.